

Especificación de Requisitos de Software (SRS) — SGB-SaaS

Sistema de Gestión Bibliotecaria Web Formato basado en ISO/IEC/IEEE 29148:2018 (Systems and software engineering — Life cycle processes — Requirements engineering).

- **Proyecto:** SGB-SaaS, Proyecto Fin de Curso (PFC), asignatura Aplicaciones Web, UTEQ (2026-2027)
- **Equipo:** Loor Medranda Marlon Taylor (Tech Lead / DevOps / Seguridad), Cajas Ibarra Irvin Marcelo (Backend), Panama Murillo Moises Antonio (Frontend)
- **Commit base de este documento:** `51607f3`
- **Repositorio:** <https://github.com/mloorm14/sgb-saas>
- **Fuente de trazabilidad:** `docs/trazabilidad/matriz.csv` (30 requisitos, validada automáticamente en CI por `scripts/validate-traceability.sh`)

Nota de método. Este documento **no redacta requisitos nuevos desde cero**: consolida y da estructura formal IEEE 29148 a lo que ya existía disperso en el repositorio antes de este commit — `docs/trazabilidad/matriz.csv` (30 requisitos con ID/tipo/prioridad/endpoint/prueba/evidencia), `docs/requisitos/historias/` y `docs/requisitos/casos-de-uso/` (formato Connextra + Cockburn), `docs/requisitos/historias-usuario.md` y `docs/requisitos/casos-de-uso.md` (los 5 HU/CU del módulo de Cajas que no siguen la convención de un archivo por HU), los 10 ADRs de `docs/adr/` y el resumen ejecutivo de `docs/informe-entrega-3.tex`. Donde el repositorio no tenía evidencia real que respalde un rationale o un criterio de aceptación, este documento lo declara explícitamente en vez de inventarlo — ver la nota de honestidad de cada requisito afectado y el resumen en la sección 6.

1. Introducción

1.1 Propósito

Este documento especifica de forma completa y verificable los requisitos funcionales y no funcionales del sistema SGB-SaaS, consolidando en un solo artefacto formal (SRS) la información que hasta esta entrega vivía correcta pero dispersa entre la matriz de trazabilidad, las historias de usuario, los casos de uso y los Architecture Decision Records (ADR). Su audiencia es el equipo de desarrollo (para verificar que la implementación actual cumple lo especificado), el evaluador del PFC (para juzgar completitud y trazabilidad), y cualquier integrante futuro del equipo que necesite entender qué se construyó y por qué, sin tener que reconstruir ese razonamiento leyendo el código o el historial de Git.

1.2 Alcance

El sistema especificado es **SGB-SaaS**: una plataforma web de gestión bibliotecaria para bibliotecas institucionales/municipales, con los módulos Auth (registro, login, logout, refresco de sesión, control de acceso por rol), Libros (catálogo con CRUD), Préstamos (creación, devolución, reportes), Reservaciones (creación, listado) y Multas (listado, pago, anulación). El alcance de este SRS cubre exactamente los **30 requisitos** ya identificados y trazados en `docs/trazabilidad/matriz.csv` al momento de este commit — no se amplía el alcance funcional del sistema al redactar este documento, solo se formaliza su especificación. Explícitamente **fuera de alcance** de este documento (y del sistema, en esta entrega): integración con sistemas académicos institucionales externos, TLS en tránsito en el entorno de desarrollo (ver REQ-NF-012), y los sub-bloques de evidencia empírica de usabilidad (SUS) que dependen de participantes humanos reales, no automatizables.

1.3 Definiciones, acrónimos y abreviaturas

Término	Significado
SRS	Software Requirements Specification (este documento)
HU / CU	Historia de Usuario / Caso de Uso
ADR	Architecture Decision Record
RBAC	Role-Based Access Control (control de acceso basado en roles)
JWT	JSON Web Token (RFC 7519)
SP	Stored Procedure (procedimiento o función almacenada en PostgreSQL)
ORM	Object-Relational Mapping (Spring Data JPA / Hibernate en este proyecto)
DTO	Data Transfer Object
TTL	Time To Live (tiempo de expiración de una entrada de cache o de una clave en Redis)
RLS	Row Level Security (PostgreSQL)
MoSCoW	Must / Should / Could / Won't — escala de priorización de requisitos
CRUD	Create, Read, Update, Delete
OWASP	Open Web Application Security Project (Top 10:2021 usado como marco de referencia de seguridad)
PFC	Proyecto Fin de Curso
UTEQ	Universidad Técnica Estatal de Quevedo
SQLSTATE	Código de error de 5 caracteres devuelto por PostgreSQL (<code>LB404</code> / <code>LB409</code> / <code>LB422</code> son códigos custom de este proyecto, ver <code>GlobalExceptionHandler</code>)

1.4 Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering (estructura de este documento).

- ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation (SQuaRE), aplicado en `docs/arquitectura/ISO25010.md`.
- OWASP Top 10:2021.
- RFC 7519 (JSON Web Token), RFC 7807 (Problem Details for HTTP APIs).
- `docs/trazabilidad/matriz.csv` — fuente primaria de los 30 requisitos.
- `docs/requisitos/historias/`, `docs/requisitos/casos-de-uso/`, `docs/requisitos/historias-usuario.md`, `docs/requisitos/casos-de-uso.md`.
- `docs/adr/ADR-001-tecnologia.md`, `ADR-003-jwt-redis.md`, `adr-006` a `adr-013` (10 ADRs).
- `docs/informe-entrega-3.tex` (resumen ejecutivo, estado del sistema, inventario de endpoints).
- `docs/arquitectura/ISO25010.md`, `docs/arquitectura/workspace.dsl` (C4).
- `docs/basedatos/CATALOGO-SP.md` (catálogo de los 7 procedimientos/funciones SQL).

1.5 Resumen del documento

La sección 2 describe el producto de forma global (perspectiva, funciones, usuarios, restricciones, supuestos). La sección 3 es el cuerpo principal: los 30 requisitos específicos, cada uno con id único, descripción, rationale, prioridad MoSCoW, criterio de aceptación medible y método de verificación. La sección 4 resume el mecanismo de trazabilidad hacia código/pruebas/evidencia. La sección 5 mapea los requisitos no funcionales contra ISO/IEC 25010. La sección 6 declara explícitamente los gaps y limitaciones honestas encontradas al consolidar este documento.

2. Descripción global

2.1 Perspectiva del producto

SGB-SaaS es un sistema nuevo (no un reemplazo ni una migración de un sistema legado), construido como PFC con una arquitectura de tres capas: frontend SPA en Angular 17, backend API REST en Spring Boot 4.0.6 (Java 21), persistencia en PostgreSQL 16, y una capa de caché/blacklist de tokens en Redis 7. El sistema modela el ciclo completo de una biblioteca institucional: catálogo de libros, préstamos, reservaciones, multas y administración de usuarios bajo RBAC. No depende de ningún sistema externo para operar (no hay integraciones con sistemas académicos institucionales ni pasarelas de pago en el alcance actual). Los cuatro servicios (frontend, backend, PostgreSQL, Redis) se orquestan con Docker Compose (ADR-012) y se comunican dentro de una red Docker interna; el único punto de entrada externo es el frontend (puerto 4200) y, para pruebas directas/Swagger, el backend (puerto 8080).

2.2 Funciones del producto

A alto nivel (el detalle completo está en la sección 3):

- **Auth:** registro de cuentas, login con emisión de JWT, logout con revocación inmediata, refresco de sesión vía cookie `HttpOnly`, bloqueo temporal tras intentos fallidos, auditoría de eventos de

autenticación, control de acceso por rol en cada endpoint.

- **Libros:** consulta paginada del catálogo, alta/edición/baja lógica de libros.
- **Préstamos:** creación (con validación de stock y estado del usuario), registro de devolución (con detección automática de atraso y generación de multa), listado de préstamos propios, reporte de libros más prestados.
- **Reservaciones:** creación (a nombre propio si es LECTOR, a nombre de otro si es BIBLIOTECARIO/GERENTE), listado por usuario.
- **Multas:** listado por usuario, pago (con desbloqueo condicional del usuario), anulación (restringida a GERENTE/ADMIN, con auditoría).

2.3 Características de los usuarios

El sistema define 4 roles (modelo RBAC normalizado, ver ADR-010 y REQ-NF-010):

Rol	Perfil de usuario típico	Conocimiento técnico esperado
LECTOR	Estudiante o miembro de la comunidad universitaria que consulta el catálogo, pide préstamos/reservaciones y ve sus propias multas.	Ninguno — usuario final de una aplicación web convencional.
BIBLIOTECARIO	Personal de mostrador que registra préstamos, devoluciones y pagos de multas presencialmente.	Bajo — debe poder operar el sistema sin soporte técnico, la guía de usabilidad (ISO 25010, "Usabilidad: Alta") asume esto explícitamente.
GERENTE	Responsable de la biblioteca; además de las funciones de BIBLIOTECARIO, puede anular multas y ver reportes.	Bajo-medio.
ADMIN	Administrador técnico/institucional del sistema; gestiona el catálogo (con la asimetría documentada en REQ-NF-010) y tiene visibilidad de auditoría.	Medio — se asume familiaridad con el dominio bibliotecario, no necesariamente con el sistema técnico subyacente.

2.4 Restricciones

- **Tecnológicas** (no negociables para esta entrega, ya decididas vía ADR): Spring Boot 4.0.6/Java 21 (ADR-001), Angular 17 (ADR-001), PostgreSQL 16 (ADR-011), Redis 7 (ADR-003/ADR-008), Docker Compose (ADR-012), Flyway 9 + `db/schema.sql / db/seed.sql` (ADR-006). Estrategia híbrida de acceso a datos obligatoria: CRUD elemental vía Spring Data JPA, operaciones multi-tabla vía procedimientos/funciones SQL (ADR-013, requisito explícito de la guía del PFC, Bloque A.2).
- **De despliegue:** el sistema debe poder levantarse completo con un solo comando (`make up / docker compose up --build`) contra un volumen de datos vacío (ADR-006, ADR-012) — requisito explícito del Bloque B de la guía (reproducibilidad).
- **De licenciamiento:** licencia MIT (ADR-009), requerida para la publicación del repositorio con DOI en Zenodo.
- **De entorno:** `JWT_SECRET` de mínimo 256 bits provisto vía `.env` (nunca hardcodeado ni committeado); ningún secreto vive en `docker-compose.yml` (ADR-012).

- **De tiempo del equipo:** 3 integrantes, sin dedicación exclusiva (proyecto académico) — restricción real que explica por qué ciertos requisitos quedan con estado "pendiente" (ver sección 6) en vez de simularse o fabricarse como completos.

2.5 Supuestos y dependencias

- Se asume que el evaluador/usuario final dispone de Docker y Docker Compose instalados (única dependencia dura del entorno de ejecución, ver README).
- Se asume disponibilidad de Redis para que el mecanismo de revocación de tokens (REQ-NF-001) y rate limiting (REQ-NF-006) funcionen; si Redis cae, `JwtAuthFilter` queda sin forma de verificar revocaciones — riesgo documentado y aceptado como pendiente de resolver para producción (ADR-003, `docs/arquitectura/ISO25010.md`, característica Fiabilidad).
- Se asume un volumen de uso de biblioteca universitaria (bajo, no concurrencia tipo e-commerce) como base para las decisiones de rendimiento — ver REQ-NF-003 y la característica "Eficiencia de desempeño" (prioridad Media) de `docs/arquitectura/ISO25010.md`.
- Este documento depende de que `docs/trazabilidad/matriz.csv` siga siendo la fuente de verdad para IDs de requisitos; si la matriz cambia (se agregan/eliminan requisitos) sin actualizar este SRS, ambos documentos se desincronizan — mismo riesgo ya documentado en ADR-006 para el par Flyway/ `schema.sql`.

3. Requisitos específicos

3.0 Convenciones usadas en cada requisito

Cada requisito **Must** incluye: **id único** (igual al de `docs/trazabilidad/matriz.csv`, para trazabilidad directa), **descripción**, **rationale** (por qué existe — basado en HU/CU/ADR reales, nunca inventado), **prioridad MoSCoW**, **criterio de aceptación medible** (derivado de los escenarios Gherkin reales de la HU/CU correspondiente cuando existen) y **método de verificación**: *Test* (prueba automatizada existente y en verde), *Demonstration* (verificado en vivo contra el stack real, con evidencia en `docs/mediciones/`), *Analysis* (decisión arquitectónica revisada por inspección/razonamiento, sin prueba automatizada de regresión), o *Inspection* (revisión directa del código fuente). Los requisitos **Should** se documentan con el mismo formato pero con menor exhaustividad cuando la fuente original (matriz/ADR) ya era menos detallada — no se rellena con contenido inventado para emparejar el formato.

3.1 Requisitos funcionales

REQ-F-001 — Registro de nuevo usuario

- **Prioridad:** Must
- **Fuente:** HU-AUTH-01, CU-AUTH-01

- **Módulo/endpoint:** `AuthController / AuthService` — `POST /api/auth/registro`
- **Descripción:** el sistema debe permitir que un visitante sin cuenta se registre con nombre, apellido, correo institucional y contraseña, quedando con rol `LECTOR` y estado `ACTIVO` por defecto.
- **Rationale:** sin registro propio, cualquier acceso al sistema dependería de que un administrador cree cada cuenta manualmente, lo cual no escala para una comunidad universitaria (HU-AUTH-01).
- **Criterio de aceptación medible:**
 1. Con correo no registrado y contraseña ≥ 8 caracteres, el sistema responde `201` con el usuario creado, rol `LECTOR`, estado `ACTIVO`, y la contraseña almacenada hasheada (nunca en texto plano).
 2. Con un correo ya registrado, el sistema responde `409` y no crea ningún usuario nuevo.
 3. Con una contraseña de menos de 8 caracteres, el sistema responde `400`.
- **Método de verificación:** **Test** parcial — `AuthServiceTest.registroCorreoDuplicado` cubre el criterio 2 (rechazo por correo duplicado). **Nota de honestidad:** la matriz señala explícitamente que este es "1 test, solo cubre el rechazo por correo duplicado, no el flujo exitoso" — los criterios 1 y 3 **no tienen prueba automatizada de regresión** en este repositorio a la fecha de este documento; se documentan como parte del comportamiento especificado (visible en el Gherkin de HU-AUTH-01) pero no como verificados por test.

REQ-F-002 — Inicio de sesión

- **Prioridad:** Must
- **Fuente:** HU-AUTH-02, CU-AUTH-02
- **Módulo/endpoint:** `AuthController / AuthService` — `POST /api/auth/login`
- **Descripción:** el sistema debe autenticar a un usuario registrado con correo y contraseña, emitiendo un `accessToken` en el cuerpo y un `refreshToken` en cookie `HttpOnly`.
- **Rationale:** es el punto de entrada de todo el control de acceso RBAC del resto del sistema (HU-AUTH-02, ADR-010).
- **Criterio de aceptación medible:**
 1. Credenciales correctas + usuario `ACTIVO` → `200` con `accessToken` y cookie `refreshToken` (`HttpOnly`, `Secure`, `SameSite=Strict`).
 2. Contraseña incorrecta → `401`, sin emitir ningún token.
 3. Usuario `BLOQUEADO_POR_MULTA` → `423`.
 4. Usuario `INACTIVO / PENDIENTE_VERIFICACION` → `403`.
- **Método de verificación:** **Test** (`AuthServiceTest.login*`, 5 tests)
 - **Demonstration** (`docs/mediciones/sec/2026-07-30-owasp-a07-fix-rate-limiting-login.md`, `docs/mediciones/sec/2026-07-30-owasp-a09-fix-logging-autenticacion.md` — verificación en vivo contra el stack Docker real).

REQ-F-003 — Cierre de sesión

- **Prioridad:** Must

- **Fuente:** HU-AUTH-03, CU-AUTH-03
- **Módulo/endpoint:** AuthController / AuthService — POST /api/auth/logout
- **Descripción:** el sistema debe invalidar de inmediato el `accessToken` de la sesión activa al cerrar sesión, aunque no haya expirado aún.
- **Rationale:** reduce la ventana de riesgo si el dispositivo queda desatendido o el token fue comprometido (HU-AUTH-03, ADR-003).
- **Criterio de aceptación medible:**
 1. Logout responde `204` .
 2. El `accessToken` usado queda en blacklist (Redis) hasta su expiración natural.
 3. Cualquier request posterior con ese mismo token es rechazado.
 4. La cookie `refreshToken` se limpia (`maxAge=0`).
 5. El evento `LOGOUT` queda registrado (correo, IP, fecha/hora).
- **Método de verificación:** **Test** (`AuthServiceTest.logoutGuardaTokenEnBlacklist`) + **Demonstration** (`docs/mediciones/sec/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-F-004 — Refresco de sesión

- **Prioridad:** Must
- **Fuente:** HU-AUTH-04, CU-AUTH-04
- **Módulo/endpoint:** AuthController / AuthService — POST /api/auth/refresh
- **Descripción:** el sistema debe emitir un `accessToken` nuevo a partir de una cookie `refreshToken` válida, sin exigir que el usuario vuelva a escribir su contraseña.
- **Rationale:** evita interrupciones de sesión cada hora (vida del `accessToken`) sin comprometer el `refreshToken` (que nunca es legible por JavaScript, ver ADR-007) — HU-AUTH-04.
- **Criterio de aceptación medible:**
 1. Cookie `refreshToken` válida presente → `200` con `accessToken` nuevo.
 2. Sin cookie `refreshToken` → `400` .
 3. `refreshToken` inválido o expirado → no se emite token nuevo (comportamiento verificado: responde `401` , no `500` , ver commit `8ce7b9e` "fix(backend): refresh con token invalido responde 401 en vez de 500").
- **Método de verificación:** **Test** (`AuthServiceTest.refreshConTokenValido`) + **Demonstration** (`docs/mediciones/sec/2026-07-21-cookie-refresh-token.md`).

REQ-F-005 — Consultar el catálogo de libros

- **Prioridad:** Must
- **Fuente:** HU-LIB-01, CU-LIB-01
- **Módulo/endpoint:** LibroController / LibroService — GET /api/v1/libros , GET /api/v1/libros/{id}

- **Descripción:** cualquier usuario autenticado (LECTOR o superior) debe poder ver el listado paginado del catálogo y el detalle de un libro.
- **Rationale:** es la operación de lectura más frecuente del sistema (HU-LIB-01, docs/arquitectura/ISO25010.md — "Eficiencia de desempeño"), de ahí también su cache Redis (REQ-NF-003).
- **Criterio de aceptación medible:**
 1. Listado paginado → 200 , ordenado por título, con stock disponible por libro.
 2. Detalle de libro existente y ACTIVO → 200 con datos completos.
 3. Libro inexistente o DADO_DE_BAJA → 404 .
- **Método de verificación:** Test (LibroServiceTest.listar_retornaPaginaDeLibros , .buscarPorId_cuandoNoExiste_lanzaEntityNotFound , LibroControllerSecurityTest , 4 tests con @PreAuthorize real vía MockMvc) + **Demonstration** (docs/mediciones/sec/2026-07-30-owasp-a01-fix-rol-admin-libros.md).

REQ-F-006 — Gestionar el catálogo de libros

- **Prioridad:** Must
- **Fuente:** HU-LIB-02, CU-LIB-02
- **Módulo/endpoint:** LibroController / LibroService — POST/PUT/DELETE /api/v1/libros{,/id}
- **Descripción:** BIBLIOTECARIO/GERENTE/ADMIN deben poder crear, editar y dar de baja (lógicamente, nunca borrado físico) libros del catálogo.
- **Rationale:** mantiene el catálogo actualizado con los ejemplares reales de la biblioteca (HU-LIB-02).
- **Criterio de aceptación medible:**
 1. ISBN nuevo + datos válidos → 201 .
 2. ISBN duplicado → 400 .
 3. Edición de libro existente → 200 con datos actualizados.
 4. Baja de libro ACTIVO → 204 , pasa a DADO_DE_BAJA (fila preservada, no borrada), deja de aparecer en listado/detalle.
 5. Rol LECTOR intentando gestionar → 403 .
- **Método de verificación:** Test (LibroServiceTest.crearLibro_cuandoIsbnNuevo_retornaDTO , .crearLibro_cuandoIsbnDuplicado_lanzaExcepcion , .eliminar_cuandoExiste_loMarcaDadoDeBaja , LibroControllerSecurityTest , 4 tests) + **Demonstration** (docs/mediciones/sec/2026-07-30-owasp-a01-fix-rol-admin-libros.md).

REQ-F-007 — Registrar préstamo

- **Prioridad:** Must
- **Fuente:** HU-01 (Cajas, en docs/requisitos/historias-usuario.md), CU-01 (docs/requisitos/casos-de-uso.md)

- **Módulo/endpoint:** `PrestamoController / PrestamoService` — `POST /api/v1/prestamos` (SP `sp_crear_prestamo`)
- **Descripción:** un BIBLIOTECARIO/GERENTE debe poder registrar el préstamo de un libro con stock disponible a un usuario `ACTIVO`, decrementando el stock en la misma transacción atómica.
- **Rationale:** núcleo del dominio bibliotecario — llevar control de qué ejemplares están fuera y cuándo deben devolverse (HU-01). La atomicidad de "crear préstamo + decrementar stock" está garantizada por el motor (`sp_crear_prestamo`), no por disciplina de código Java (ADR-013).
- **Criterio de aceptación medible:**
 1. Libro con stock > 0 y usuario `ACTIVO` → préstamo `ACTIVO`, stock decrementado en 1.
 2. Libro sin stock → `422` ("sin stock disponible"), sin crear registro.
 3. Usuario `BLOQUEADO_POR_MULTA` → `422` ("multas pendientes"), sin crear registro.
 4. Usuario o libro inexistente → `404`.
- **Método de verificación: Test**
(`PrestamoServiceTest.crear_conDatosValidos_invocaProcedimientoYRetornaDTO`, `PrestamoMultaProcedureIntegrationTest` — 6 tests de integración reales contra PostgreSQL, no mocks) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md`).

REQ-F-008 — Registrar devolución

- **Prioridad:** Must
- **Fuente:** HU-02 (Cajas), CU-02
- **Módulo/endpoint:** `PrestamoController / PrestamoService` — `POST /api/v1/prestamos/{id}/devolucion` (SP `sp_registrar_devolucion`)
- **Descripción:** registrar la devolución de un préstamo activo, incrementando el stock del libro y generando una multa automáticamente si hubo atraso.
- **Rationale:** liberar stock y detectar atraso sin intervención manual del bibliotecario (HU-02); la atomicidad de hasta 4 tablas en una sola transacción es exactamente el caso que justifica usar un SP en vez de ORM puro (ADR-013).
- **Criterio de aceptación medible:**
 1. Devolución sin atraso → préstamo `DEVUELTO`, stock +1, sin multa.
 2. Devolución con atraso → préstamo `DEVUELTO`, multa `PENDIENTE` generada, usuario pasa a `BLOQUEADO_POR_MULTA`.
 3. Doble devolución del mismo préstamo → `409`.
- **Método de verificación: Test**
(`PrestamoServiceTest.registrarDevolucion_sinAtraso_noGeneraMulta`, `.registrarDevolucion_conAtraso_generaMulta`, `PrestamoMultaProcedureIntegrationTest`, 3 tests) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md`).

REQ-F-009 — Ver préstamos propios

- **Prioridad:** Must
- **Fuente:** HU-F02 (Panama), CU-F02
- **Módulo/endpoint:** `PrestamoController / PrestamoService` + `PrestamosLectorComponent` — GET `/api/v1/prestamos/usuario/{id}, .../activos`
- **Descripción:** un LECTOR debe poder ver sus propios préstamos (activos e históricos) desde la interfaz, sin poder consultar los de otro usuario.
- **Rationale:** autoservicio de información sin depender del mostrador (HU-F02); el aislamiento por usuario es un caso concreto de control de acceso, no solo una preferencia de UX.
- **Criterio de aceptación medible:**
 1. LECTOR autenticado ve sus propios préstamos con fecha límite.
 2. LECTOR que intenta pedir los préstamos de otro usuario → acceso denegado.
 3. Sin préstamos registrados → mensaje explícito, no tabla vacía sin contexto (criterio de UI, ver HU-F02).
- **Método de verificación: Test**
(`PrestamoServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado` , `prestamos-lector.component.spec.ts` , 2 tests) + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2e.md`).

REQ-F-010 — Reporte de libros más prestados

- **Prioridad:** Should
- **Fuente:** **sin HU/CU dedicada** — la matriz marca explícitamente `historia_usuario` y `caso_de_uso` como — para este requisito.
- **Módulo/endpoint:** `PrestamoController / PrestamoService` — GET `/api/v1/prestamos/reportes/libros-mas-prestados` (función `fn_reporte_libros_mas_prestados`)
- **Descripción:** exponer un reporte de los libros con más préstamos registrados, con un límite configurable (default 10).
- **Rationale:** **nota de honestidad** — no hay una HU/CU que documente la necesidad de negocio detrás de este reporte; se infiere que sirve para decisiones de adquisición/gestión del catálogo (rol GERENTE), pero esa motivación no está respaldada por un documento de requisitos específico, solo por la existencia de la función SQL y su test. No se fabrica un rationale más elaborado del que el repositorio realmente sostiene.
- **Criterio de aceptación medible:** sin límite explícito en el request, el sistema aplica un default de 10 resultados (único comportamiento con test de regresión).
- **Método de verificación: Test**
(`PrestamoServiceTest.reporteLibrosMasPrestados_sinLimite_aplicaDefaultDiez`).

REQ-F-011 — Crear reservación

- **Prioridad:** Must
- **Fuente:** HU-03 (Cajas) + HU-F03 (Panama), CU-03 (Cajas) + CU-F03 (Panama)
- **Módulo/endpoint:** `ReservacionController / ReservacionService` + `ReservacionesComponent` — `POST /api/v1/reservaciones`
- **Descripción:** un usuario autenticado debe poder reservar un libro; si es LECTOR, siempre a su propio nombre (se ignora cualquier `usuarioId` distinto enviado en el request); si es BIBLIOTECARIO/GERENTE, puede reservar a nombre de otro usuario.
- **Rationale:** asegurar un ejemplar sin stock disponible en el momento (HU-03/HU-F03); la resolución del usuario destino ignorando el `usuarioId` del body para un LECTOR es un control de acceso deliberado (mismo patrón que REQ-NF-011 para el rol ejecutor en anulación de multas).
- **Criterio de aceptación medible:**
 1. LECTOR reserva → reservación `PENDIENTE` a su propio nombre, con fecha de reserva = ahora y fecha límite de retiro calculada.
 2. BIBLIOTECARIO/GERENTE reserva a nombre de otro usuario → reservación a nombre del usuario indicado.
 3. LECTOR que envía un `usuarioId` distinto al propio → el sistema lo ignora, la reservación se crea igual a su propio nombre.
 4. Libro inexistente → `404`.
- **Método de verificación:** `Test (ReservacionServiceTest.crear_*, 4 tests; reservaciones.component.spec.ts, 2 tests)` + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2e.md`).

REQ-F-012 — Listar reservaciones propias

- **Prioridad:** Must
- **Fuente:** HU-F03 (Panama, **inferida** — la matriz señala explícitamente que el mismo componente cubre creación y listado, sin una HU dedicada solo a listar), CU-F03
- **Módulo/endpoint:** `ReservacionController / ReservacionService` — `GET /api/v1/reservaciones/usuario/{id}`
- **Descripción:** un usuario debe poder listar sus propias reservaciones, con el mismo aislamiento por usuario que REQ-F-009.
- **Rationale: nota de honestidad** — no existe una HU separada para "listar reservaciones"; se infiere del hecho de que `ReservacionesComponent` (frontend) implementa ambas operaciones (creación y listado) y de que existe un test de servicio dedicado al aislamiento por usuario. Se documenta como inferido, no como si existiera una HU explícita que no existe.
- **Criterio de aceptación medible:** un LECTOR que intenta pedir las reservaciones de otro usuario recibe acceso denegado (único comportamiento con test de regresión para este requisito específico).
- **Método de verificación:** `Test (ReservacionServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado)`

- **Demonstration** (docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2e.md).

REQ-F-013 — Ver multas propias

- **Prioridad:** Must
- **Fuente:** HU-F01 (Panama), CU-F01
- **Módulo/endpoint:** `MultaController / MultaService + MultasComponent` — GET `/api/v1/multas/usuario/{id}`
- **Descripción:** un LECTOR debe poder ver el detalle de sus multas (monto, fecha, estado) sin poder pagarlas ni anularlas desde la UI.
- **Rationale:** autoservicio de información sin exponer acciones reservadas a otros roles (HU-F01) — el lector ve un mensaje indicando que debe acercarse a la biblioteca para regularizar, en vez de un botón de pago que de todas formas el backend rechazaría.
- **Criterio de aceptación medible:**
 1. LECTOR ve su multa `PENDIENTE` con monto, fecha y estado.
 2. La UI del lector no muestra botones "Pagar"/"Anular".
 3. LECTOR que intenta pedir las multas de otro usuario → acceso denegado.
- **Método de verificación: Test**
(`MultaServiceTest.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado` , `multas.component.spec.ts` , 2 tests) + **Demonstration** (docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2e.md).

REQ-F-014 — Pagar multa

- **Prioridad:** Must
- **Fuente:** HU-04 (Cajas), CU-04
- **Módulo/endpoint:** `MultaController / MultaService` — POST `/api/v1/multas/{id}/pago` (SP `sp_pagar_multa`)
- **Descripción:** un BIBLIOTECARIO/GERENTE debe poder registrar el pago de una multa `PENDIENTE` ; si era la última multa pendiente del usuario, este vuelve a estado `ACTIVO` .
- **Rationale:** el lector recupera la posibilidad de pedir préstamos solo cuando ya no tiene ninguna multa pendiente (HU-04); el desbloqueo condicional (verificar que no queden otras multas) es exactamente el tipo de lógica multi-fila que justifica un SP (ADR-013).
- **Criterio de aceptación medible:**
 1. Pago de la única multa pendiente → multa `PAGADA` , usuario `ACTIVO` .
 2. Pago con otras multas pendientes → multa pagada cambia a `PAGADA` , usuario permanece `BLOQUEADO_POR_MULTA` .
- **Método de verificación: Test** (`MultaServiceTest.pagar_invocaProcedimientoYRetornaDTO` , `.pagar_conOtrasMultasPendientes_noDesbloqueaUsuario` , `PrestamoMultaProcedureIntegrationTest.pagarMulta_unicaPendiente_desbloqueaUsuario`)

- **Demonstration** (docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md).

REQ-F-015 — Anular multa

- **Prioridad:** Must
- **Fuente:** HU-05 (Cajas), CU-05
- **Módulo/endpoint:** MultaController / MultaService — POST /api/v1/multas/{id}/anulacion (SP sp_anular_multa)
- **Descripción:** solo GERENTE/ADMIN pueden anular una multa registrada por error o por excepción justificada, quedando auditado quién tomó la decisión.
- **Rationale:** corregir sin perder trazabilidad de quién autorizó la excepción (HU-05); el rol ejecutor se resuelve **únicamente** desde la sesión autenticada, nunca desde el body del request — defensa en profundidad reforzada también a nivel de SP (LB422 si el rol no es válido), no solo en el controller (HU-AUTH-07, REQ-NF-010/011).
- **Criterio de aceptación medible:**
 1. GERENTE/ADMIN anula multa PENDIENTE → multa ANULADA , fila nueva en bitacora_auditoria .
 2. BIBLIOTECARIO intenta anular → 403 antes de llegar al procedimiento.
 3. BIBLIOTECARIO que envía "rolEjecutor": "GERENTE" en el body → el campo se ignora completamente, la operación igual se rechaza.
- **Método de verificación: Test**
(MultaServiceTest.anular_conRolGerente_resuelveRolDesdeAuthentication , .anular_conRolAdmin_resuelveRolAdmin , .anular_sinRolGerenteOAdmin_lanzaAccesoDenegado , PrestamoMultaProcedureIntegrationTest.anularMulta_* , 2 tests) + **Demonstration** (docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md).

REQ-F-016 — Gestión de préstamos y devoluciones desde la interfaz

- **Prioridad:** Must
- **Fuente:** HU-F04 (Panama), CU-F04
- **Módulo/endpoint:** PrestamosGestionComponent (frontend), reutiliza los mismos endpoints de REQ-F-007/REQ-F-008
- **Descripción:** el bibliotecario debe poder crear un préstamo y registrar su devolución desde la interfaz web, sin depender de anotaciones manuales.
- **Rationale:** capa de UI sobre la lógica ya especificada en REQ-F-007/REQ-F-008 (HU-F04) — no introduce reglas de negocio nuevas, solo la superficie de interacción.
- **Criterio de aceptación medible:**
 1. Bibliotecario crea préstamo con usuario, libro y días → préstamo registrado.
 2. Bibliotecario registra devolución de un préstamo activo → fila se actualiza con fecha real, botón de devolución desaparece de esa fila.
 3. Préstamos ya devueltos no muestran botón de devolución.

4. Rechazo del backend (ej. sin stock) → mensaje de error sin cerrar el formulario.

- **Método de verificación:** **Test** (`prestamos-gestion.component.spec.ts` , 2 tests, capa UI sin acceso directo a BD) + **Demonstration** (`docs/mediciones/frontend/2026-07-30-flujo-frontend-prestamos-reservaciones-multas-e2e.md`).

3.2 Requisitos no funcionales

Clasificados según las categorías de ISO/IEC/IEEE 29148 aplicables a este proyecto: rendimiento, seguridad, y calidad de software/arquitectura. La gran mayoría de los NF de este sistema son de seguridad porque el foco real de esta entrega (Bloque C.2 de la guía) fue una auditoría OWASP Top 10 en vivo, no una elección arbitraria de énfasis de este documento.

3.2.1 Rendimiento

REQ-NF-003 — TTL configurable del cache del catálogo

- **Prioridad:** Should
- **Fuente:** sin HU dedicada (requisito de configuración), CU-LIB-01, ADR-008
- **Módulo:** `LibroService` — `GET /api/v1/libros`
- **Descripción:** el cache Redis del listado de libros debe expirar tras un TTL declarado en configuración externa (`CACHE_LIBROS_TTL_SECONDS` , default 300s), no hardcodeado en Java.
- **Rationale:** antes de esta decisión no existía TTL alguno (cache infinito, solo invalidado manualmente vía `@CacheEvict`) — riesgo real si los datos subyacentes cambiaran por una vía que el backend no controla (ADR-008).
- **Criterio de aceptación medible:** la clave `libros::SimpleKey []` en Redis expira según el TTL configurado, confirmado con `redis-cli TTL` contra el contenedor real (no simulado).
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/2026-07-21-cache-libros-ttl.md` , TTL confirmado en vivo; latencia medida ~170ms en cache miss vs ~30ms en cache hit según `docs/arquitectura/ISO25010.md`). **Nota:** la prueba de carga formal del Bloque C.1 (k6, 50 VUs) que mide este mismo endpoint bajo concurrencia se ejecutó en un prompt posterior a la redacción original de este requisito — ver `docs/mediciones/perf/REPORT.md` (p95 caliente 29.79ms, p95 frío 7.96ms, ambos dentro de umbral).

3.2.2 Seguridad

REQ-NF-001 — Revocación inmediata de tokens (blacklist)

- **Prioridad:** Must
- **Fuente:** HU-AUTH-03, CU-AUTH-03, ADR-003

- **Descripción:** todo `accessToken` invalidado por logout debe quedar en una blacklist de Redis hasta su expiración natural.
- **Rationale:** JWT stateless no tiene revocación nativa — sin esto, un token robado o una sesión cerrada seguiría siendo válido hasta expirar por sí solo (ADR-003, OWASP A07).
- **Criterio de aceptación medible:** clave `blacklist:<jti>` existe en Redis con TTL igual al tiempo restante de expiración del token; una request posterior con ese token es rechazada.
- **Método de verificación:** **Test** (`AuthServiceTest.logoutGuardaTokenEnBlacklist`) + **Demonstration** (`docs/mediciones/sec/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-NF-002 — Cookie `HttpOnly/Secure/SameSite` para el refresh token

- **Prioridad:** Must
- **Fuente:** HU-AUTH-04, ADR-007
- **Descripción:** el `refreshToken` debe transportarse exclusivamente en una cookie `HttpOnly`, `Secure`, `SameSite=Strict`, con `path=/api/auth`, nunca en el cuerpo JSON.
- **Rationale:** un secreto de vida larga (7 días) legible por JavaScript es un vector directo de exfiltración vía XSS; migrarlo a cookie `HttpOnly` lo hace inaccesible a JS por diseño del navegador (ADR-007, OWASP A02). **Nota de honestidad heredada de ADR-007:** el `accessToken` (de vida corta, 1h) **no** está migrado a cookie todavía — sigue en el cuerpo JSON/memoria del frontend, decisión explícitamente diferida por el impacto en `jwt.interceptor.ts` / `auth.service.ts`.
- **Criterio de aceptación medible:** la respuesta de login/refresh incluye el header `Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Strict; Path=/api/auth`; el campo `refreshToken` está ausente del cuerpo JSON (`@JsonIgnore` en `TokenResponseDTO`).
- **Método de verificación:** **Demonstration** (`docs/mediciones/sec/2026-07-21-cookie-refresh-token.md`, verificado con `curl --include` contra el stack real).

REQ-NF-006 — Rate limiting de intentos de login

- **Prioridad:** Must
- **Fuente:** HU-AUTH-05, CU-AUTH-05, `LoginRateLimiter`
- **Descripción:** una combinación correo+IP debe bloquearse temporalmente (429) tras 5 intentos fallidos consecutivos en 900s.
- **Rationale:** dificultar fuerza bruta sin abrir una vía para que un atacante bloquee a la víctima usando su correo desde otra IP — precisamente por eso la clave es correo+IP, no solo correo (HU-AUTH-05, OWASP A07). Este control nació de un hallazgo real de la auditoría de seguridad de esta entrega (ausencia total de rate limiting), no era un requisito preexistente.
- **Criterio de aceptación medible:**
 1. 6.º intento fallido desde el mismo correo+IP en 15 min → 429, sin validar la contraseña.
 2. El bloqueo es por correo+IP: la víctima real, desde su propia IP, no está bloqueada aunque el atacante haya fallado contra su correo desde otra IP.
 3. Un login exitoso resetea el contador de esa combinación a cero.

- **Método de verificación:** **Test** (`LoginRateLimiterTest` , 6 tests; `AuthServiceTest.login*RateLimit*` , 3 tests) + **Demonstration** (`docs/mediciones/sec/2026-07-30-owasp-a07-fix-rate-limiting-login.md`).

REQ-NF-007 — Auditoría de eventos de autenticación

- **Prioridad:** Must
- **Fuente:** HU-AUTH-06, CU-AUTH-06
- **Descripción:** todo `LOGIN_OK` , `LOGIN_FAIL` y `LOGOUT` debe quedar registrado con IP, fecha/hora y usuario/correo, consultable en logs de aplicación y en `bitacora_auditoria` .
- **Rationale:** permitir investigar un incidente de seguridad después de ocurrido, sin depender de que alguien lo reporte en el momento (HU-AUTH-06, OWASP A09). Igual que REQ-NF-006, corrige un hallazgo real de la auditoría (ausencia total de logging de autenticación antes de esta entrega).
- **Criterio de aceptación medible:**
 1. Login exitoso → evento `LOGIN_OK` con IP, timestamp, id de usuario.
 2. Login fallido → evento `LOGIN_FAIL` con correo intentado, IP, timestamp (sin id de usuario, porque nunca se resolvió).
 3. Logout → evento `LOGOUT` con correo, IP, timestamp.
- **Método de verificación:** **Test** (`AuthServiceTest.loginExitosoReseteaContadorDeRateLimit` , `.loginFallidoIncrementaContadorDeRateLimit` , `.logoutGuardaTokenEnBlacklist` — verifican `bitacoraAuditoriaRepository.save()`) + **Demonstration** (`docs/mediciones/sec/2026-07-30-owasp-a09-fix-logging-autenticacion.md`).

REQ-NF-010 — RBAC aplicado consistentemente con defensa en profundidad

- **Prioridad:** Must
- **Fuente:** HU-AUTH-07, CU-AUTH-07, ADR-010
- **Descripción:** cada endpoint debe verificar el rol del usuario únicamente desde su sesión autenticada, aplicado tanto vía `@PreAuthorize` (Spring Security) como, para las operaciones críticas vía SP, una segunda verificación en el propio procedimiento SQL.
- **Rationale:** ningún usuario debe poder ejecutar una acción reservada a otro rol, ni manipulando el request (HU-AUTH-07); la verificación duplicada (aplicación + base de datos) es defensa en profundidad deliberada, no redundancia accidental (ADR-010, OWASP A01).
- **Criterio de aceptación medible:**
 1. Rol no autorizado en un endpoint restringido → `403` antes de ejecutar lógica de negocio.
 2. Si la verificación de la capa de aplicación se saltara, el SP (ej. `sp_anular_multa`) igual rechaza con `SQLSTATE LB422` .
- **Método de verificación:** **Test** (`MultaServiceTest.anular_sinRolGerenteOAdmin_lanzaAccesoDenegado` , `.listarPorUsuario_cuandoLectorPideOtroUsuario_lanzaAccesoDenegado` + patrones equivalentes en `PrestamoServiceTest` / `ReservacionServiceTest`)

- **Demonstration** (docs/mediciones/sec/2026-07-30-owasp-a01-control-acceso-roto.md). **Nota de honestidad** (ya documentada en el informe técnico): existe una asimetría real entre controllers — `LibroController` incluye `ADMIN` en sus 5 endpoints, `PrestamoController` / `ReservacionController` no, verificado en vivo al construir docs/postman/coleccion.json .

REQ-NF-011 — El rol ejecutor nunca se resuelve desde el body del request

- **Prioridad:** Must
- **Fuente:** HU-AUTH-07, `AuthorizationDeniedException` handler
- **Descripción:** el rol usado para autorizar una acción debe resolverse siempre desde el JWT de la sesión (`Authentication`), nunca desde un campo del cuerpo del request (ej. un hipotético `"rolEjecutor"`).
- **Rationale:** cerrar la vía trivial de escalamiento de privilegios que existiría si el backend confiara en cualquier dato de rol enviado por el cliente (HU-AUTH-07, OWASP A01).
- **Criterio de aceptación medible:** un usuario que envía un campo de rol falsificado en el body sigue siendo autorizado/rechazado según su rol real de sesión, no según el valor enviado.
- **Método de verificación: Test**
(`MultaServiceTest.anular_sinRolGerenteOAdmin_lanzaAccesoDenegado`) + **Demonstration**
(docs/mediciones/sec/2026-07-30-owasp-a01-control-acceso-roto.md).

REQ-NF-012 — TLS en tránsito

- **Prioridad:** Should
- **Fuente:** sin HU dedicada — decisión de entorno, OWASP A02
- **Descripción:** las comunicaciones cliente-servidor deberían viajar cifradas (HTTPS) en el entorno de producción/evaluación final.
- **Rationale:** proteger credenciales y tokens en tránsito frente a observación de red (OWASP A02).
- **Estado real — pendiente, declarado sin ambigüedad:** el entorno de desarrollo actual corre sobre HTTP plano **por diseño** — la guía del PFC exige TLS recién en la Entrega Final, no en esta. **No existe evidencia real de que este requisito esté implementado hoy;** no se fabrica un criterio de aceptación "cumplido" que no se pueda sostener.
- **Criterio de aceptación medible (para cuando se implemente):** toda petición HTTP sin TLS a un endpoint protegido debe redirigirse o rechazarse; el certificado debe validarse sin advertencias en el navegador.
- **Método de verificación: Analysis** (pendiente de implementación; no aplica Test/Demonstration todavía) — docs/mediciones/sec/2026-07-30-owasp-a02-fallo-criptografico.md documenta el hallazgo y el estado pendiente.

REQ-NF-013 — Prevención de inyección SQL

- **Prioridad:** Must
- **Fuente:** sin HU dedicada, OWASP A03

- **Descripción:** toda consulta (ORM o SP) debe ser parametrizada, sin concatenación de SQL con datos de entrada del usuario.
- **Rationale:** la inyección SQL es uno de los riesgos más severos y mejor entendidos de OWASP Top 10; el patrón híbrido de este proyecto (JPA parametrizado + SPs con parámetros tipados) lo cubre por construcción en ambos mecanismos (ADR-013, OWASP A03).
- **Criterio de aceptación medible:** ninguna consulta del código fuente concatena directamente un valor de entrada del usuario dentro de una cadena SQL.
- **Método de verificación: Analysis** — verificación manual puntual durante la auditoría original (docs/mediciones/sec/2026-07-30-owasp-a03-inyeccion.md), **sin test de regresión permanente en el suite** (la propia matriz lo señala explícitamente con un ☐ en la columna de prueba automatizada) — se documenta la ausencia de un test de regresión en vez de implicar que existe uno.

REQ-NF-014 — Cabeceras de seguridad / Content-Security-Policy

- **Prioridad:** Should
- **Fuente:** sin HU dedicada, OWASP A05
- **Descripción:** el frontend/backend deberían enviar cabeceras de seguridad estándar (incluyendo CSP) para mitigar XSS y clickjacking.
- **Estado real — pendiente, declarado sin ambigüedad:** igual que REQ-NF-012, este requisito queda explícitamente **no implementado** en esta entrega (ver docs/mediciones/sec/2026-07-30-owasp-a05-mala-configuracion-seguridad.md), con SecurityConfig / frontend-angular/nginx.conf identificados como los puntos donde se implementaría a futuro.
- **Criterio de aceptación medible (para cuando se implemente):** las respuestas HTTP incluyen Content-Security-Policy , X-Frame-Options / frame-ancestors , y el resto de cabeceras OWASP Secure Headers recomendadas.
- **Método de verificación: Analysis** (pendiente de implementación).

3.2.3 Calidad de software / arquitectura

REQ-NF-004 — Estrategia híbrida de acceso a datos (ORM + SP)

- **Prioridad:** Must
- **Fuente:** HU-01 (Cajas, lado SP) + HU-AUTH-06 (Marlon, lado ORM), ADR-013
- **Descripción:** el CRUD elemental de una sola tabla debe implementarse vía Spring Data JPA; cualquier operación con joins, agregaciones o transacción atómica multi-tabla debe implementarse como procedimiento/función SQL.
- **Rationale:** requisito explícito de la guía del PFC (Bloque A.2), no una preferencia de estilo — ver el análisis completo de alternativas descartadas (ORM puro, SP puro) en ADR-013.
- **Criterio de aceptación medible:** los 7 objetos SQL catalogados en docs/basedatos/CATALOGO-SP.md cubren exactamente las operaciones multi-tabla; el resto del acceso a datos usa JpaRepository estándar.

- **Método de verificación:** **Test** (`PrestamoMultaProcedureIntegrationTest` , 6 tests contra PostgreSQL real; `AuthServiceTest` , 8 tests contra el lado ORM) + **Demonstration** (`docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md`).

REQ-NF-005 — Esquema de base de datos reproducible

- **Prioridad:** Should
- **Fuente:** decisión arquitectónica, ADR-006
- **Descripción:** un evaluador debe poder levantar el sistema completo con datos ya poblados usando un solo comando, sin ejecutar migraciones manualmente.
- **Rationale:** requisito explícito del Bloque B de la guía (reproducibilidad automática); Flyway sigue siendo la fuente de verdad incremental, `db/schema.sql` + `db/seed.sql` es el snapshot de conveniencia para inicialización desde cero (ADR-006).
- **Criterio de aceptación medible:** `docker compose down -v && make up` reconstruye el stack completo (26 tablas, datos de ejemplo) desde un volumen vacío, sin pasos manuales adicionales.
- **Método de verificación:** **Demonstration** — verificado en vivo repetidamente durante esta entrega (ver Status de ADR-006 y ADR-012).

REQ-NF-008 — PostgreSQL como motor único de base de datos

- **Prioridad:** Should
- **Fuente:** decisión arquitectónica, ADR-011
- **Descripción:** el sistema usa PostgreSQL 16 como único motor de base de datos, con Row Level Security para aislar datos por rol.
- **Rationale:** RLS nativo (sin el cual el aislamiento por lector dependería de disciplina de código en cada endpoint), PL/pgSQL maduro para los 7 objetos SQL, integridad referencial estricta sobre un dominio intrínsecamente relacional — ver comparación completa contra MySQL/MongoDB en ADR-011.
- **Criterio de aceptación medible:** las 26 tablas, 7 procedimientos/funciones y las políticas RLS de `db/roles-privilegios.sql` corren contra un contenedor `postgres:16-alpine` real.
- **Método de verificación:** **Test** (`PrestamoMultaProcedureIntegrationTest` corre contra PostgreSQL real, no un mock) + **Analysis** (revisión de la decisión arquitectónica en sí, ADR-011).

REQ-NF-009 — Despliegue vía Docker Compose

- **Prioridad:** Should
- **Fuente:** decisión arquitectónica, ADR-012
- **Descripción:** los 4 servicios del sistema se orquestan con Docker Compose, con imágenes base pinadas por digest sha256 y healthchecks que ordenan el arranque.
- **Rationale:** reproducibilidad de un solo comando sin la complejidad operativa de un orquestador pensado para escalado multi-nodo que este proyecto no necesita (ADR-012, comparación completa contra Kubernetes y despliegue manual).

- **Criterio de aceptación medible:** `docker compose ps` reporta los 4 servicios como `healthy` / `Up` tras `make up`; las imágenes base están documentadas por digest en `docs/DIGESTS-LOG.md`.
 - **Método de verificación: Demonstration** — verificado en vivo repetidamente (Status de ADR-012).
-

3.3 Requisitos de interfaz externa

El sistema expone una única interfaz externa real: una **API REST sobre HTTP/JSON**, documentada automáticamente vía `springdoc-openapi` (Swagger UI en `/swagger-ui.html`, ver ADR-001) y consumida por el frontend Angular. No existen requisitos de interfaz externa con ID propio en `docs/trazabilidad/matriz.csv` — cada endpoint concreto ya está trazado como parte del requisito funcional que lo usa (columna `endpoint_api` de la matriz, sección 3.1 de este documento). El inventario completo de los 19 endpoints REST reales (verificado línea por línea contra los 5 `@RestController`, no inferido) vive en `docs/informe-entrega-3.tex` (sección "Estado del sistema") y en `docs/postman/coleccion.json` (39 requests reales, incluyendo casos de error 401/403/404/422/423/429). Este SRS no duplica esa tabla; remite a ambas fuentes para el detalle exacto de método, ruta y rol requerido por endpoint.

Contrato general: request/response en JSON; autenticación vía header `Authorization: Bearer <accessToken>` (excepto `/api/auth/refresh`, que usa la cookie `refreshToken`); errores en formato `ProblemDetail` (RFC 7807) vía `GlobalExceptionHandler`, sin fuga de detalles internos (stacktraces, mensajes de motor de base de datos) al cliente.

4. Trazabilidad

La trazabilidad completa de los 30 requisitos hacia historia de usuario, caso de uso, módulo/endpoint, prueba automatizada, tipo de acceso a datos, evidencia empírica y estado vive en `docs/trazabilidad/matriz.csv`, validada automáticamente en cada ejecución de CI por `scripts/validate-traceability.sh` (ver `ci(trazabilidad)`: agrega `scripts/validate-traceability.sh` y lo integra a CI, commit `6c351cf`). Este SRS no reemplaza esa matriz — la expande en prosa formal IEEE 29148 (rationale, criterio de aceptación medible, método de verificación explícito) mientras la matriz sigue siendo la fuente machine-readable para validación automática. Si un requisito nuevo se agrega al sistema a futuro, el proceso correcto es: (1) agregar la fila a `matriz.csv`, (2) expandir la entrada correspondiente en este SRS, en ese orden — nunca solo uno de los dos, por el mismo riesgo de desincronización ya documentado en ADR-006 para Flyway/ `schema.sql`.

5. Requisitos de calidad de software (ISO/IEC 25010)

`docs/arquitectura/ISO25010.md` ya mapea las 8 características de calidad de producto de ISO/IEC 25010 contra escenarios concretos de SGB-SaaS y la estrategia que atiende cada una, con prioridad asignada por

criterio del equipo (no medición, salvo donde se cita evidencia real). Este SRS no duplica esa tabla completa; la resume aquí para dejar explícita la relación con los requisitos no funcionales de la sección 3.2:

Característica ISO 25010	Prioridad	Requisito(s) NF relacionado(s)
Adecuación funcional	Alta	REQ-F-007, REQ-F-008, REQ-NF-004
Eficiencia de desempeño	Media	REQ-NF-003
Compatibilidad	Media	3.3 (interfaz REST/JSON)
Usabilidad	Alta	REQ-F-016, REQ-F-013 (mensajes explícitos en UI)
Fiabilidad	Alta	REQ-NF-001 (riesgo fail-open/fail-closed de Redis, pendiente)
Seguridad	Alta	REQ-NF-001, 002, 006, 007, 010, 011, 012 (pendiente), 013, 014 (pendiente)
Mantenibilidad	Alta	10 ADRs de docs/adr/ , docs/basedatos/CATALOGO-SP.md
Portabilidad	Alta	REQ-NF-005, REQ-NF-009

6. Notas de honestidad y gaps conocidos (resumen)

Consolidado de todas las notas de honestidad ya señaladas en línea en la sección 3, para que quien audite este documento no tenga que buscarlas una por una:

1. **REQ-F-001**: solo 1 de 3 criterios de aceptación tiene prueba automatizada de regresión (rechazo por correo duplicado); el flujo exitoso y el rechazo por contraseña corta no tienen test.
2. **REQ-F-010**: sin HU/CU que documente su motivación de negocio; el rationale de este SRS es una inferencia razonable, no un hecho documentado previamente.
3. **REQ-F-012**: sin HU dedicada; se infiere de la implementación (mismo componente que REQ-F-011).
4. **REQ-NF-002**: el `accessToken` sigue sin migrar a cookie `HttpOnly` (solo el `refreshToken` lo está) — gap real y deliberadamente diferido, no un olvido.
5. **REQ-NF-010**: asimetría real de roles entre `LibroController` (incluye ADMIN) y `PrestamoController` / `ReservacionController` (no lo incluyen).
6. **REQ-NF-012 y REQ-NF-014**: declarados explícitamente **pendientes**, sin criterio de aceptación fabricado como si estuvieran cumplidos.
7. **REQ-NF-013**: verificado por inspección manual puntual durante la auditoría original, sin test de regresión permanente en el suite.
8. **HU/CU de Cajas (HU-01 a HU-05, CU-01 a CU-05)**: viven consolidadas en `docs/requisitos/historias-usuario.md` y `docs/requisitos/casos-de-uso.md`, no como un archivo por HU/CU como el resto de módulos (`docs/requisitos/historias/` , `docs/requisitos/casos-de-uso/`) — inconsistencia real de convención de archivos entre módulos, documentada aquí en vez de normalizada silenciosamente (normalizarla sería una tarea de refactor de documentación fuera del alcance de este SRS).
9. **ADRs**: el resumen ejecutivo de `docs/informe-entrega-3.tex` corrige una cifra de "13 ADRs" a los 10 reales existentes en `docs/adr/` — este SRS usa consistentemente la cifra real (10).

10. **Diagrama de clases UML:** docs/observaciones/OBSERVACIONES.md (OBS-02) ya documenta que este diagrama sigue sin versionar como imagen en el repositorio — no es un gap de este SRS, es un gap heredado y ya reportado en su propia bitácora de observaciones.